

COMMON COURSE ASSIGNMENT FORMAT

NAME: SK. ASHWAQ AHAMED (192572070)

A. Assignment Information

Department	COMPUTER SCIENCE ENGINEERING
Programme	B.Tech
Course Code & Course Name	CSA0924 – Programming in Java
Academic Year / Batch	2025
Faculty Name	Padmakala S
Assignment Title	Real-Time Concurrent Bank Fund-Transfer & Ledger System — Preventing the “Double-Spend / Lost-Update” Bug
Date of Issue	01/09/2026
Date of Submission	01/09/2026
Maximum Marks	100
Course Outcome(s) – CO	CO1, CO2, CO3, CO5 (core) and CO4 (bonus)
Bloom's Taxonomy Level	L3–L6 (Apply, Analyze, Evaluate, Create)
SDG Mapping (SDG 1-17)	SDG 8 – Decent Work and Economic Growth (resilient financial infrastructure)
Industry / Societal Relevance	Lost-update and double-spend defects caused by unsynchronized concurrent access to shared balances are real, recurring production incidents in banking, fintech, and payment-gateway systems; this assignment reproduces the exact class of concurrency defect financial-systems engineers.

1. Executive Summary

This report presents the design and validation of a concurrency-safe digital banking core in Java. The assignment focuses on a production-style lost-update/double-spend defect in which multiple simultaneous withdrawal or transfer requests can pass the same balance check before any request updates the shared balance. It also addresses deadlocks caused by two threads locking the same pair of accounts in opposite order.

Testing compared unsafe and fixed implementations. The supplied report records an unsafe final balance of ₹-800.00, while the synchronized implementation produced ₹0.00. The naive two-account locking test produced a deadlock, whereas consistent account-number lock ordering completed successfully. JDBC validation showed a ₹500 transfer changing A101 from ₹10,000.00 to ₹9,500.00 and A102 from ₹10,000.00 to ₹10,500.00.

2. Assignment Overview

Course	CSA09 – Programming in Java
Assignment Title	Real-Time Concurrent Bank Fund-Transfer & Ledger System — Preventing the “Double-Spend / Lost-Update” Bug
CO Mapping	CO1, CO2, CO3, CO5 (core); CO4 (bonus)
Bloom’s Level	L3–L6 (Apply, Analyze, Evaluate, Create)
SDG Mapping	SDG 8 – Decent Work and Economic Growth
Core Technologies	Java, Collections, Multithreading, Exceptions, JDBC, MySQL, Java I/O
Concurrent Test	20 withdrawal threads; shared small account pool
Database Test	Accounts A101/A102; ₹10,000 opening balance; ₹500 transfer

3. Problem Statement and Understanding

The digital bank experiences a critical concurrency defect during peak load. Multiple requests may target the same account within milliseconds. Each request independently checks whether sufficient funds exist, but without synchronization several threads can observe the same old balance and subsequently debit it. This violates the intended atomic check-then-debit operation and can produce a negative or otherwise incorrect final balance.

A second problem occurs during transfers. A transfer must coordinate both the source and destination accounts. If one thread locks A101 and waits for A102 while another locks A102 and waits for A101, circular waiting occurs and both threads can remain blocked indefinitely.

3.1 Expected Outcomes

- Account balances must never become negative.
- No valid balance update may be lost under concurrent execution.
- Two-account transfers must not deadlock.

- A transfer must either fully succeed or fully fail.
- Failed operations must use the required custom exceptions and restore state after a partial failure.
- Successful transactions and updated balances must be persisted through JDBC.
- Banking activity must be recorded in an audit log.

3.2 Test Inputs Used

- Accounts: A101 and A102.
- Initial database balance: ₹10,000 for each account.
- Concurrent withdrawal threads: 20.
- JDBC transfer amount: ₹500.
- Rollback demonstration initial balance: ₹1,000.

4. Application of Course Knowledge

4.1 Object-Oriented Design (CO1)

The banking domain is represented using Bank, Customer, Transaction, Account, SavingsAccount, and CurrentAccount classes. SavingsAccount and CurrentAccount extend Account and use method overriding for account-specific interest/fee behaviour. Encapsulation is applied by keeping balance private and allowing changes only through controlled account operations.

- Inheritance: SavingsAccount and CurrentAccount derive from Account.
- Polymorphism: account-specific applyInterestOrFee() behaviour can be invoked through the Account abstraction.
- Encapsulation: balance is not exposed as a public mutable field.
- Abstraction: banking operations are expressed through domain-level methods instead of direct balance manipulation.

4.2 Collection Framework (CO2)

A Map<String, Account> is used as the account directory, keyed by account number. A List<Transaction> is used as the running transaction ledger. Generics provide type safety, and iterators can be used to traverse collections without exposing their internal representation.

Collection	Use	Reason
HashMap	Account lookup	Fast average-case lookup and simple prototype usage.
ConcurrentHashMap	Concurrent account directory	Useful when map-level concurrent access is required.
TreeMap	Sorted account lookup	Maintains key ordering but adds logarithmic lookup cost.

4.3 Multithreading and Concurrency (CO3)

The project uses Java threads/executor services to simulate at least 20 concurrent banking requests. The unsafe implementation intentionally omits synchronization to reproduce the defect. The corrected

implementation protects the check-and-debit sequence so that the balance check and update behave as one atomic operation from the perspective of competing threads.

For transfers, both accounts must be protected. The final design uses a consistent lock-ordering rule based on account number: the account with the smaller account number is locked first. This removes the circular-wait condition created by arbitrary lock acquisition order.

4.4 Custom Exception Handling

Exception	Purpose
InsufficientBalanceException	Rejects a withdrawal/transfer when sufficient funds are unavailable.
InvalidAccountException	Reports an invalid or missing account.
AccountFrozenException	Prevents operations on a frozen account.
TransactionLimitExceededException	Rejects a transaction that exceeds the configured limit.

5. Solution Design and Methodology

5.1 Development Stages

1. Create the domain model and account hierarchy.
2. Create the account directory and transaction ledger using generic collections.
3. Implement the unsafe withdrawal/transfer path and run the concurrent stress test.
4. Introduce synchronization or explicit locking to make balance updates atomic.
5. Demonstrate naive two-account deadlock and then apply consistent lock ordering.
6. Implement custom checked exceptions and rollback after partial transfer failure.
7. Connect the Java application to MySQL using JDBC and PreparedStatement.
8. Persist successful transactions and updated balances, and generate audit logs.
9. Validate the system using before/after console and database evidence.

5.2 Synchronization Strategy Comparison

Strategy	Strengths	Trade-offs	Decision
synchronized methods/blocks	Simple; automatic monitor release; easy to reason about.	Less explicit control over locking behaviour.	Preferred for the prototype.
ReentrantLock	Explicit lock/unlock; tryLock and additional control.	Requires careful lock management and unlock handling.	Considered as an alternative.

5.3 Deadlock-Avoidance Strategy

Arbitrary locking was rejected because it allows opposite-direction transfers to create circular waiting. Consistent account-number ordering was selected because every transfer obtains multiple locks in the same global order. Therefore, one thread cannot hold the higher-numbered lock while waiting for the

lower-numbered lock while another thread holds the lower-numbered lock and waits for the higher-numbered lock.

5.4 Rollback Strategy

The transfer design treats a transfer as an all-or-nothing operation. If a failure occurs after the debit succeeds, the debit is reversed so that the source account returns to its original balance. The supplied validation records a rollback demonstration in which a ₹300 debit temporarily changed R101 from ₹1,000.0 to ₹700.0, after which the failure triggered rollback and restored R101 to ₹1,000.0.

6. Tools and Technologies Used

Tool	Purpose
Java JDK 26.0.1	Compiling and executing Java programs
Visual Studio Code	Writing and managing Java source files
MySQL Workbench	Creating and checking the database
MySQL	Persistent storage for accounts and transactions
MySQL Connector/J 26.7.0	JDBC connectivity between Java and MySQL
Git/GitHub	Source-code and version-control management
Java I/O	Maintaining the audit log

7. Database Persistence with JDBC (CO5)

The Java application connects to MySQL through JDBC. Successful deposits, withdrawals, and transfers are recorded in the transactions table, while account balances are updated in persistent storage. A mini-statement can be obtained by querying recent transaction records.

PreparedStatement is used instead of string-concatenated SQL. Parameterized queries separate SQL structure from user-supplied values, improve safety against SQL injection, and make repeated statements easier to manage.

7.1 JDBC Validation

Stage	Account	Customer	Balance
Before transfer	A101	Rahul	₹10,000.00
Before transfer	A102	Priya	₹10,000.00
After ₹500 transfer	A101	Rahul	₹9,500.00
After ₹500 transfer	A102	Priya	₹10,500.00

The validation also records that the transaction was inserted into the transactions table. A failed transfer scenario is required to leave database balances unchanged.

8. Results and Validation

8.1 Before Fix – Unsynchronized Withdrawal

The 20-thread unsafe test produced the following recorded result:

UNSAFE FINAL BALANCE = ₹-800.00

This demonstrates the lost-update/double-spend defect: multiple threads were able to pass the balance check before the shared balance was safely updated.

8.2 After Fix – Synchronized Withdrawal

FIXED FINAL BALANCE = ₹0.00

The corrected implementation rejected excess withdrawal requests with an insufficient-balance condition and prevented the negative-balance outcome.

8.3 Deadlock Demonstration

The naive implementation produced the following recorded evidence:

- pool-3-thread-1 locked A101
- pool-3-thread-2 locked A102
- DEADLOCK DETECTED

After applying lock ordering, the recorded result was:

All lock-ordered transfers completed = true

A101 = ₹10000.00 and A102 = ₹10000.00 were recorded after the lock-ordered transfer stress test. The consistent ordering prevents the circular wait.

8.4 Rollback Validation

- Debit succeeded. R101 = ₹700.0
- Failure detected → rollback performed.
- R101 after rollback = ₹1000.0
- Original balance = ₹1000.0

The result confirms that the source balance was restored after the simulated failure.

9. Analysis and Engineering Decision

The unsafe and corrected results clearly show why shared monetary state cannot be modified using an unprotected check-then-update sequence. The unsafe run reached ₹-800.00, whereas the synchronized run reached the expected ₹0.00. This provides direct before/after evidence for the concurrency fix.

For synchronization, synchronized methods/blocks were selected for the prototype because they are straightforward and automatically release the monitor when execution exits the protected region.

ReentrantLock was considered because it provides explicit control and features such as tryLock, but that control also increases the responsibility for correct lock management.

For deadlock avoidance, arbitrary lock acquisition was rejected because the stress test reproduced the classic opposite-direction deadlock. Account-number ordering was selected because it establishes one global lock order and removes circular waiting. The lock-ordered test completed successfully.

The implementation is therefore a suitable prototype for demonstrating Java concurrency concepts. For a production multi-server banking system, JVM-level synchronization alone would not be sufficient because different servers would not share the same Java monitor.

10. Broader Considerations and Professional Responsibility

A concurrency defect in financial software can cause direct financial loss when customers can spend or transfer more than their legitimate balance. It can also produce inaccurate ledgers, customer disputes, regulatory exposure, investigation costs, and loss of trust.

Financial applications therefore require rigorous testing of concurrent code paths before release. Transaction data should be protected against unauthorized access and modification, and significant financial events should be auditable. These considerations support the assignment's emphasis on thread safety, rollback, persistence, and audit logging.

11. Scalability, Security and Limitations

The prototype runs within a single Java process. If the system were extended to multiple application servers sharing one database, concurrency control would need to move beyond JVM-level synchronization. Database-level row locking such as SELECT ... FOR UPDATE, optimistic concurrency control, or distributed transaction/saga approaches could be considered depending on the architecture.

Security is addressed conceptually through controlled access to account and transaction data, parameterized SQL, and audit logging. Encryption and full authentication/authorization are outside the demonstrated scope.

The main limitation is that the assignment implementation is a prototype. It does not implement a complete payment network, KYC process, production authentication model, or a distributed deployment.

12. Conclusion

The project successfully demonstrates how Java can be used to design a concurrency-safe banking transfer and ledger core. The unsynchronized implementation reproduced the lost-update/double-spend problem, while the synchronized implementation prevented the negative-balance outcome.

The project also reproduced a two-account deadlock using naive lock acquisition and resolved it with consistent account-number lock ordering. The rollback test restored the original balance after a simulated partial failure. JDBC integration with MySQL persisted account and transaction data, and Java I/O was used for audit logging.

Overall, the implementation meets the central prototype requirements: atomic balance updates, no negative balance in the corrected test, deadlock prevention, custom exception handling, rollback, JDBC persistence, and auditability.

13. Student Reflection

13.1 What I Learned

I learned that multithreaded programs can produce incorrect financial results even when each individual operation appears logically correct. The assignment helped me understand atomic check-then-update operations, synchronization, deadlock prevention, rollback, Java collections, and JDBC-based persistence.

13.2 Future Improvements

- Add the optional Swing/AWT teller dashboard with live balances and a transaction counter.
- Strengthen database transaction management and use database-level locking where appropriate.
- Add stronger authentication and authorization controls.
- Create automated unit, integration, and concurrency stress tests.
- Evaluate the design across multiple application servers to study distributed concurrency.

14. Requirements Validation Summary

Requirement	Evidence / Result	Status
OOP domain model	Banking classes, inheritance, polymorphism, encapsulation	Met
Collections	Map<String, Account> and List<Transaction>	Met
Concurrent load	20 concurrent withdrawal threads	Met
No negative balance	Fixed run: ₹0.00; excess requests rejected	Met
Deadlock prevention	Lock ordering; all transfers completed	Met
Custom exceptions	Four required checked exceptions specified/used	Met
Rollback	R101 restored from ₹700.0 to ₹1000.0	Met
JDBC persistence	MySQL connection, balance update and transaction record	Met

RESULT AND VALIDATION

Present: console output/log excerpts showing the negative-balance/lost-update bug occurring in the unsynchronized version; console/thread-dump excerpts showing the deadlock occurring in the naive two-account-lock version; console output/log excerpts showing correct, deadlock-free behaviour in the fixed version; before/after screenshots of the database table; and, if attempted, a screenshot of the GUI teller dashboard. Validate explicitly whether the stated requirements (no negative balance, no lost update, no deadlock, correct rollback on partial failure) are satisfied.

DashboardFund TransferDeposit / WithdrawAccountsTransaction LedgerConcurrency TestsSystem Output

TOTAL ACCOUNTS
Accounts: 4

TOTAL BALANCE
Total Balance: ₹43000.00

TRANSACTIONS
Transactions: 0

BANKING SYSTEM FEATURES

1. Object-Oriented account model

2. Savings and Current Account inheritance

3. Collections Framework using Map and List

4. Concurrent fund transfers

5. ReentrantLock based synchronization

6. Lost-update / double-spend demonstration

7. Deadlock demonstration

8. Deadlock prevention using lock ordering

9. Custom checked exceptions

10. Transaction rollback after partial failure

11. Transaction ledger

12. Java I/O audit logging

Use the tabs above to perform banking operations and run the concurrency demonstrations.

DashboardFund TransferDeposit / WithdrawAccountsTransaction LedgerConcurrency TestsSystem Output

From Account:

A101

To Account:

A102

Amount:

500

Account				
A101	Rahul	Savings	₹11000.00	ACTIVE
A102	Priya	Savings	₹10000.00	ACTIVE
A103	Arun	Current	₹15000.00	ACTIVE
A104	Meena	Current	₹8000.00	ACTIVE

15. References

- Oracle Java Documentation – Java language, collections, concurrency, and exception-handling concepts.
- Oracle JDBC Documentation – JDBC API and database connectivity.
- MySQL Documentation – database, transaction, and locking concepts.
- MySQL Connector/J Documentation – JDBC driver for MySQL.
- Visual Studio Code Documentation – Java development environment.
- Git Documentation – version control.
- OpenAI ChatGPT – AI-assisted support for understanding Java, JDBC, concurrency, and project documentation.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

G. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation	CO1	[Faculty to map]	L3/L4	10
Application of knowledge	CO1, CO2	[Faculty to map]	L3/L4	20
Solution / Design / Implementation	CO1, CO2, CO3	[Faculty to map]	L4/L5/L6	20
Modern tool usage (multithreading + JDBC)	CO3, CO5	[Faculty to map]	L3/L4	10
Validation	CO3, CO5	[Faculty to map]	L4/L5	15
Analysis & justification	CO3	[Faculty to map]	L4/L5	15
Broader considerations	CO1–CO5	[Faculty to map]	L4/L5	5
Documentation & reflection	CO1–CO5	[Faculty to map]	L3/L4	5

H. Faculty Evaluation & Continuous Improvement

CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: _____

Areas requiring improvement: _____

Corrective / Improvement Action: _____

Action to be implemented in the next offering: _____